

APÉNDICE 1

EJERCICIOS



1.1. Introducción

1.1.1. Presentación de los enunciados

En este apéndice se proponen ejercicios con diversos niveles de dificultad. Dada la gran cantidad de ejercicios, los mismos están catalogados según su nivel de dificultad y su importancia de resolverlo.

- **Dificultad:** Básica, Moderada, Intermedia, Alta, Extrema.
- **Requerido:** Indispensable, Recomendable.

1.1.2. Modo de trabajo

Para cada uno de los siguientes ejercicios:

1. Determinar cuáles son los datos de entrada, contexto y salida.
2. Clasificar según su multiplicidad y tipo de procesamiento (vertical, horizontal o ambos).
3. Diseñar el algoritmo que resuelve el ejercicio, diagramarlo, codificarlo y probar su correcto funcionamiento.

1.2. Lección 1

1.2.1. Operaciones aritméticas

Dificultad: básica, *Requerido:* Indispensable.

Se ingresan por teclado dos valores numéricos enteros y positivos, a y b , se pide calcular e informar por consola el resultado de las siguientes operaciones:

1. La suma: $a+b$.
2. La diferencia: $a-b$.
3. El producto: $a*b$.
4. El cociente: a/b , aceptando que b es distinto de 0 (cero).

1.2.2. Cociente entre dos números

Dificultad: básica, *Requerido:* Indispensable.

Se ingresan por teclado dos valores numéricos enteros: a y b , se pide calcular e informar el cociente a/b , validando que b sea distinto de cero. En tal caso, mostrar un mensaje de error en la consola.

1.2.3. División entera y módulo

Dificultad: básica, *Requerido:* Indispensable.

Se ingresa por teclado un valor numérico entero, informar:

1. La quinta parte de dicho valor.
2. El resto que surge al dividir el valor ingresado en cinco partes iguales.
3. La séptima parte de la quinta parte del valor ingresado.

1.2.4. Mayor valor entre dos números

Dificultad: básica, *Requerido:* Indispensable.

Se ingresa por teclado dos valores numéricos enteros diferentes entre sí, informar cuál es el mayor.

1.2.5. Mayor y menor valor entre dos números

Dificultad: básica, *Requerido:* Indispensable.

Se ingresa por teclado dos valores numéricos enteros, informar cuál es el mayor y cuál el menor, y si son iguales se debe mostrar un mensaje con el siguiente texto: “Los valores ingresados son iguales”.



Tip! Programación APT (A Prueba de Tontos)

En la jerga de los programadores, existe el concepto de programar APT o No APT (A Prueba de Tontos); referido a si corresponde (o no) que el programador se ocupe de validar que el ingreso de datos coincida con lo esperado.

Si programamos APT aceptamos que el usuario es *un tonto* capaz de ingresar letras donde esperamos números, o valores negativos donde esperamos números positivos. Por eso, preparamos el programa para validar casos de este tipo, y eventualmente mostraremos los mensajes de error que correspondan.

En cambio, si programamos No APT, nos desentendemos de la responsabilidad de validar el ingreso de los datos, porque aceptamos que *el usuario no es ningún tonto*. Si le pedimos que ingrese valores diferentes entre sí, lo hará; si le pedimos un número entero suponemos que no ingresará una cadena de caracteres, etcétera.

En este curso, siempre trabajaremos No APT. No validaremos ningún caso de error, salvo que el enunciado del ejercicio lo requiera explícitamente.

Sin embargo, en la vida real sí debemos programar A Prueba de Tontos.

1.2.6. Mayor, medio y menor valor entre tres números

Se ingresan tres valores numéricos enteros, diferentes entre sí, informar cuál es el menor, cuál está en el medio y cuál es el mayor.

1.2.7. Tipo de triángulo según sus lados

Dificultad: **básica**, *Requerido:* **Indispensable**.

Se ingresan tres valores que representan las longitudes de los lados de un triángulo, informar cuál es el tipo del triángulo ingresado: isósceles, equilátero o escaleno.

1.2.8. Separar los atributos de una fecha

Dificultad: **básica**, Requerido: **Indispensable**.

Dado un número de ocho dígitos que representa una fecha con formato *aaaammdd*, se pide mostrar por separado el día, el mes y el año de la fecha ingresada.



Tip! Usar los operadores / (división) y % (módulo)

Ejemplo:

```
int f = 20260423;    // 2026/04/23
int anio = f/10000;   // anio vale: 2026
int dia = f%100       // dia vale: 23
```

1.2.9. Unificar los atributos de una fecha

Dificultad: **básica**, Requerido: **Indispensable**.

Se ingresan tres valores numéricos enteros que indican el día, mes y año de una fecha. Se pide unificarlos para obtener un valor numérico entero de ocho dígitos, que represente dicha fecha con el siguiente formato: *aaaammdd*, siendo los primeros cuatro dígitos el año, los dos siguientes el mes y los dos últimos el día.

1.2.10. Fecha más próxima

Dificultad: **moderada**, Requerido: **Indispensable**.

Entre dos fechas indicadas por el usuario, informar cuál es la más cercana a la actual.

Se debe establecer:

1. Cuáles son los datos de entrada que el algoritmo necesitará para resolver el problema planteado.
2. En qué formato el usuario deberá ingresar dichos datos de entrada.

Se debe considerar los años bisiestos, entendiendo que un año es bisiesto si es divisible por 4, o por 400 pero no por 100.



Tip! Obtener el valor absoluto de un número

La función `abs`, contenida en la biblioteca `stdlib.h`, retorna el valor absoluto del valor que recibe como parámetro.

```
int a = -1;
int b = abs(a); // b vale: 1
```

1.2.11. Cuántos días tiene un mes

Dificultad: **básica**, Requerido: **Indispensable**.

Se ingresan dos valores numéricos enteros que representan mes y año de una fecha, siendo mes=1 para enero, mes=2 para febrero, etcétera. Se pide informar cuántos días tiene el mes. Pregunta: ¿es realmente necesario el año en este ejercicio?

1.2.12. Producto mediante sumas sucesivas

Dificultad: **básica**, Requerido: **Indispensable**.

Dados dos valores numéricos enteros, informar su producto calculándolo por sumas sucesivas.

1. Considerando que los valores ingresados serán números positivos o cero.
2. Considerando que los valores ingresados también podrían ser negativos.

1.2.13. Factorial de un número

Dificultad: **básica**, Requerido: **Indispensable**.

Dado un valor numérico entero positivo, informar su factorial.

NOTA: El factorial de un número n (se indica $n!$) se calcula así: $n * n-1 * n-2 * \dots * 3 * 2 * 1$. El factorial de 0 es 1. Por ejemplo: 5! es: 120, 4! es: 24 y 0! es: 1.

1.2.14. Números primos

Dificultad: **básica**, Requerido: **Indispensable**.

Dado un valor entero positivo, informar si es un número primo.

NOTA: Un número es primo si sólo es divisible por sí mismo y por 1.

1.2.15. Primeros números primos

Dificultad: básica, *Requerido:* Indispensable.

Dado un valor numérico entero positivo n , informar los primeros n números primos. Por ejemplo: si n fuera 6, entonces la salida debería ser: 1, 2, 3, 5, 7, 11. Si n fuera 3, la salida sería: 1, 2, 3.

1.2.16. Número de Fibonacci

Dificultad: básica, *Requerido:* Indispensable.

Dado un valor numérico entero positivo n , informar el n -ésimo término de la sucesión de Fibonacci. Por ejemplo: si n es 6, la salida del programa debe ser 8.

NOTA: Los primeros dos términos de la serie de Fibonacci son 1 y 1. Luego, cada término se calcula como la suma de los dos términos anteriores. Así, los primeros términos de la serie de Fibonacci son: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, etcétera.

1.2.17. Múltiplos

Dificultad: básica, *Requerido:* Indispensable.

Considerar que cada punto de este ejercicio es un ejercicio en sí mismo.

- Dado un valor numérico entero positivo n , informar los primeros n múltiplos de 5 que no sean múltiplos de 3. Por ejemplo: Si n fuera 6, la salida deberá ser: 5, 10, ~~15~~, 20, 25, ~~30~~, 35, 40.
- El usuario ingresa tres valores: a , b y n . Mostrar los primero n múltiplos de a que no sean múltiplos de b .
- Ídem anterior, pero comenzando desde el múltiplo más grande. Por ejemplo: si el usuario ingresa $a=5$, $b=3$ y $n=6$, la salida será: 40, 35, 25, 20, 10, 5.

1.2.18. Factorial de los primeros números naturales

Dificultad: básica, *Requerido:* Indispensable.

Dado un valor numérico entero positivo n , se pide calcular e imprimir el factorial de los primeros n números naturales. Por ejemplo: si n fuera 7, entonces la salida deberá ser: 1, 2, 6, 24, 120, 720, 5040.

1.2.19. Suma de fracciones

Dificultad: **básica**, Requerido: **Indispensable**.

El usuario ingresa dos valores que representan el numerador y denominador de una fracción. Luego, ingresa dos valores más que representan el numerador y denominador de otra fracción. Se pide mostrar la fracción que resulta de sumar las dos fracciones ingresadas por el usuario.

1.2.20. Simplificar una fracción

Dificultad: **básica**, Requerido: **Indispensable**.

El usuario ingresa dos valores que representan el numerador y denominador de una fracción. Se pide mostrar la fracción equivalente simplificando, tanto como sea posible la fracción que el usuario ingresó. Por ejemplo: $18/12 \rightarrow 3/2$.



Ejercicios de múltiples registros

Los ejercicios con los que hemos estado trabajando son ejercicios de *registro único*, pues en todos los casos se le solicitó al usuario ingresar un único registro compuesto por una pequeña cantidad de valores.

A partir de aquí, comenzamos a resolver ejercicios de múltiples registros, que requerirán leer y procesar individualmente los datos de cada uno de los registros de entrada que el usuario ingresará.

Por ejemplo, en un ejercicio cuyo enunciado comienza diciendo: “Se ingresa un conjunto de valores numéricos enteros positivos, que finaliza cuando se ingresa un valor 0 (cero) o negativo...”, utilizaremos una estructura como la siguiente para procesar los valores del conjunto que el usuario ingresará:

```
int v;
cin >> v;

while( v>=0 )
{
    // :
    // procesar el valor leído...
    // :
```

```
cin >> v;
}
```

La variable v tomará, uno a uno, los valores del conjunto a medida que el usuario los vaya ingresando.

Si el enunciado dijese: “El usuario ingresará un conjunto de pares de valores numéricos, enteros y positivos, que finaliza cuando se ingrese un par con ambos valores negativos...”, la estructura que nos permitirá el conjunto de todos los pares de valores (registros) que ingresará el usuario será la siguiente:

```
int a,b;

cin >> a >> b;

while( a>0 && b>0 )
{
    // :
    // procesar registro leído (par de valores)...
    // :

    cin >> a >> b;
}
```

1.2.21. Docenas

Dificultad: [básica](#), *Requerido:* [Indispensable](#).

Se ingresan por teclado varios valores enteros positivos, de a uno por vez. Se solicita informar:

1. Cuántos valores v fueron ingresados, tal que $v \leq 12$.
2. Cuántos valores v fueron ingresados, tal que $12 < v \leq 24$.
3. Cuántos valores v fueron ingresados, tal que $24 < v \leq 36$.
4. Cuántos valores v fueron ingresados, tal que $v == 0$ (cero).

La carga de datos finaliza cuando se ingresa un valor negativo.

1.2.22. Cantidades, promedios y porcentajes

Dificultad: básica, *Requerido:* Indispensable.

Se ingresan varios valores numéricos enteros, finalizando la carga de datos al ingresar un 0 (cero). Se pide informar:

1. Cantidad positivos.
2. Cantidad de negativos.
3. Porcentaje de pares.
4. Promedio de los positivos.
5. Porcentaje de negativos.

1.2.23. Mayores y menores que

Dificultad: básica, *Requerido:* Indispensable.

Dados 50 valores numéricos enteros, que se ingresan de a uno por vez, se pide informar el promedio de los mayores que 100, y la suma de los menores que -10.

1.2.24. Máximos

Dificultad: básica, *Requerido:* Indispensable.

Cada uno de los siguientes ítems debe considerarse como un ejercicio en sí mismo.

1. Dados 100 valores enteros positivos, informar cuál es el mayor.
2. Dados 100 valores enteros, todos mayores que -10, informar cuál es el mayor.
3. Dados 100 valores enteros, informar cuál es el mayor.

1.2.25. Mínimos

Dificultad: básica, *Requerido:* Indispensable.

Cada uno de los siguientes ítems debe considerarse como un ejercicio en sí mismo.

1. Dados 100 valores enteros negativos, informar cuál es el menor.
2. Dados 100 valores enteros menores que 10, informar cuál es el menor.
3. Dados 100 valores enteros, informar cuál es el menor.

1.2.26. Mayor de los negativos, menor de los positivos

Dificultad: **moderada**, Requerido: **Indispensable**.

Dado un conjunto de valores numéricos que finaliza con el ingreso de un 0 (cero), informar cuál es el mayor de los negativos, y cuál el menor de los positivos.

1.2.27. Mínimo valor dentro de un intervalo

Dificultad: **moderada**, Requerido: **Indispensable**.

Sea un conjunto de valores numéricos que finaliza al ingresar un 0 (cero), informar cuál es el mínimo valor considerando sólo aquellos que pertenecen al intervalo $[-16, 27]$.

1.2.28. Persona más joven, persona más vieja

Dificultad: **moderada**, Requerido: **Indispensable**.

Se ingresa un conjunto de pares (n, f) , donde n es el nombre de una persona y f su fecha de nacimiento expresada como *aaaammdd*, informar el nombre de la persona más joven y el de la más vieja.

1.2.29. Conjuntos y subconjuntos

Dificultad: **moderada**, Requerido: **Indispensable**.

Se ingresan n conjuntos de m valores numéricos cada uno. Se pide informar:

1. Para cada uno de los n conjuntos:
 - a. El valor promedio.
 - b. El máximo valor.
 - c. Porcentaje de valores positivos.
2. Para todo el lote de datos:
 - a. Valor promedio.
 - b. Porcentaje de valores negativos.
 - c. Valor mínimo.

1.2.30. Conjunto dividido por valores 0 (cero)

Dificultad: **moderada**, Requerido: **Indispensable**.

Se dispone de un conjunto de valores enteros positivos cuyo ingreso finaliza con la llegada de un número negativo.

El conjunto está dividido en subconjuntos, separados entre sí mediante valores 0 (cero). Se pide informar:

1. Por cada subconjunto:
 - a. Promedio de sus valores.
 - b. Valor mínimo.
2. Para el conjunto completo:
 - a. Cantidad de subconjuntos.
 - b. Sumatoria de sus valores.
 - c. Número del subconjunto en el que se ingresó el mayor valor (será único), indicando también cuál fue ese valor y en qué posición relativa se encontró.

1.2.31. Seguidilla

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Se ingresa un conjunto de valores numéricos enteros que finaliza con la llegada de un 0 (cero), se pide informar:

- a. Cuántas veces se ingresaron valores consecutivos ascendentes. A esto, en adelante, lo llamaremos *seguidilla*.
- b. Por cada seguidilla, qué cantidad de elementos la componen.
- c. En qué posición relativa aparece la seguidilla más larga.

Por ejemplo, supongamos que los datos que ingresa el usuario son los siguientes:

5 3 **1 4 6 8** 3 **1 4 5 6 7 9** **4 6 8** 4 2 **1 3 5 6 7** 2 0

La salida que deberá arrojar el programa será:

- a. Cantidad de seguidillas: 4.
- b. 4 elementos (seguidilla **azul**), 6 elementos (seguidilla **verde**), 3 elementos (seguidilla **roja**), 5 elementos (seguidilla **violeta**).

- c. La seguidilla más larga se ingresó en la posición relativa: 2.



Tip! Tipo de dato *char*

El tipo `char` permite declarar variables preparadas para contener un carácter. Por ejemplo: `char c='A'`. En este caso, la variable `c` contiene el carácter `A`.

Veamos el siguiente ejemplo:

```
cout << "Ingrese la inicial de su nombre:" << endl;
char c;

cin >> c; // el usuario ingresa un caracter
cout << "Su nombre comienza con la letra: " << c << endl;
```

1.2.32. Palabras dentro de una oración

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Se ingresa una serie de caracteres que conforman las palabras de una oración. Cada palabra está separada de la siguiente por medio de un carácter ' ' (espacio). La oración finaliza cuando se ingresa un ' . '.

Se pide informar:

- Cantidad de veces que apareció cada vocal.
- Cantidad de palabras que contiene la oración.
- Cantidad de letras que posee la palabra más larga.

Además, indicar si la longitud de las palabras que componen la oración es creciente; ejemplo: "Sí que siempre conseguís maravillosas oportunidades".

NOTA: Los caracteres de la oración deben ingresarse uno por uno a través del teclado. Si se utiliza Eclipse o VS Code es probable que no pueda ingresar el carácter ' ' (espacio); entonces, se recomienda asumir que las palabras de la oración están separadas entre sí por medio de carácter ' _ ' (guion bajo).

1.3. Lección 2

1.3.1. Función factorial

Dificultad: básica, *Requerido:* Indispensable.

Desarrollar y probar la función `factorial`, que calcula y retorna el factorial de n .

```
double factorial(int n);
```

1.3.2. Factorial de los primeros n números

Dificultad: básica, *Requerido:* Indispensable.

Usando la función `factorial`, se pide mostrar el factorial de los primeros n números naturales, siendo n un valor que el usuario ingresará por consola.

1.3.3. Función esPrimo

Dificultad: básica, *Requerido:* Indispensable.

Desarrollar y probar la función `esPrimo`, que retorna `true` o `false` según determine que n es un número primo o no:

```
bool esPrimo(int n);
```

1.3.4. Primeros n números primos

Dificultad: moderada, *Requerido:* Indispensable.

Usando la función `esPrimo` desarrollar un programa que muestre los primeros n números primos, siendo n un valor que el usuario ingresará por consola.

1.3.5. Función Fibonacci

Dificultad: moderada, *Requerido:* Indispensable.

Desarrollar y probar la función `fibonacci`, que calcula y retorna el n -ésimo término de la sucesión de Fibonacci.

```
double fibonacci(int n);
```

1.3.6. Primeros términos de la sucesión de Fibonacci

Dificultad: moderada, *Requerido:* Indispensable.

- a. Mostrar los primeros n términos de la sucesión de Fibonacci, siendo n un valor que el usuario ingresará por consola.
- b. Mostrar los primeros n términos primos de la sucesión de Fibonacci, siendo n un valor que el usuario ingresará por consola.
- c. Mostrar el factorial de los primeros n términos de la sucesión de Fibonacci, siendo n un valor que el usuario ingresará por consola.

1.4. Lección 3

Los ejercicios de esta lección deben resolverse dentro del proyecto `AlgoritmosAFondo_EMPTY`, (o proyecto AAFE), siguiendo los lineamientos establecidos en el documento de Especificaciones y API (o documento EYA).

Antes de comenzar, es necesario contar con el mencionado documento (AYE), y descargar e importar el proyecto AAFE en Visual Studio Code. Ambos recursos están disponibles a través de los dos primeros códigos QR.

El tercer código QR permite acceder a una *playlist* de videos en YouTube cuyos videos muestran, entre otras cosas, cómo importar el proyecto AAFE en Visual Studio Code y cómo usar el documento EYA.



[Descargar documento
Especificaciones Y API](#)



[Descargar Algoritmos
AFondo_EMPTY](#)



[Cómo importar el
proyecto y modalidad
de trabajo](#)

1.4.1. TAD Fracción

Dificultad: **moderada**, *Requerido:* **Indispensable**.

1. Analizar el TAD `Fraccion`, que ya está documentado y desarrollado en EYA.

2. Ubicar el archivo `Fraccion.hpp` en el proyecto. Se encuentra en la carpeta `biblioteca/tads/intro`.
3. Completar la estructura `struct Fraccion` y codificar cada una de las funciones del TAD copiando las implementaciones que se muestran en EYA.
4. Probar que todo funciona correctamente codificando el siguiente `main`.

```
int main()
{
    Fraccion a = fraccion(2,9);
    cout << fraccionToString(a) << endl; // debo ver 2/9

    Fraccion b = fraccion(4,6);
    Fraccion c = fraccionSumar(a,b);
    cout << fraccionToString(c) << endl; // debo ver 48/54

    Fraccion d = fraccionSimplificar(c);
    cout << fraccionToString(d) << endl; // debo ver 8/9

    return 0;
}
```

5. Agregar al TAD `Fraccion` las siguientes funciones, implementarlas y probar su funcionamiento en el programa `main`.

```
Fraccion fraccionMultiplicar(Fraccion f,int escalar);
```

```
Fraccion fraccionMultiplicar(Fraccion f,Fraccion g);
```

1.4.2. Fechas, horas y milisegundos

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Usando las funciones de la biblioteca de *Gestión de Instantes En Milisegundos*, documentadas en EYA, entre las que se encuentran:

- `currTimeMillis`,
- `attributesToMillis`,
- `millisToAttributes` y
- `millisToString`.

Implementar la estructura y las funciones del TAD `Fecha` (incluido en el proyecto AAFE, en `biblioteca/tads/partel`), según los lineamientos y especificaciones indicadas en el documento. Probarlo, por ejemplo, así:

```
int main()
{
    Fecha f1 = fecha();
    Fecha f2 = fecha(18,3,2026);
    int dias = fechaDiffDias(f1,f2);

    cout << "Entre " << fechaToString(f1) << " y ";
    cout << fechaToString(f2) << " transcurrieron ";
    cout << dias << " dias." << endl;

    return 0;
}
```

1.4.3. TAD Timer

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Implementar el TAD `Timer` dentro del archivo `Timer.hpp`, que se ubica en la carpeta `biblioteca/tads/partel`, dentro del proyecto AAFE, cuya funcionalidad permite medir el tiempo que transcurre entre dos instantes. La documentación, así como las especificaciones, se encuentran disponibles en EYA.

Un ejemplo de uso podría ser el siguiente:

```
int main()
{
    Timer t = timer();
    cout << "Presiona ENTER..." << endl;
    string x;

    timerStart(t);
    cin >> x;
    timerStop(t);

    int sec = timerElapsedTime(t)/1000;
    cout << "Demoraste " << sec << " segundos" << endl;

    return 0;
}
```

```
};
```

1.4.4. Medición de tiempos

Dificultad: **moderada**, Requerido: **Indispensable**.

1.4.4.1. Probar cuánto tiempo demora un doble while

Utilizar el TAD `Timer` para medir cuánto demora la computadora en resolver la función `f`, para los siguientes valores de n : 100, 1000, 10000, 100000.

```
void f(int n)
{
    int i=0;
    while(i<n)
    {
        int j=0;
        while(j<n)
        {
            j = j+1;
        }

        i = i+1;
    }
}
```

1.4.4.2. Cuánto demora calcular un número de Fibonacci

Usando la siguiente implementación recursiva del número de Fibonacci, calcular cuánto tiempo podría requerir resolver `fibonacci(100)`.

Sugerencia: medir cuánto demora para $n=40$, $n=41$, ..., $n=50$, e inferir.

```
double fibonacci(int n)
{
    if(n<2)
    {
        return 1;
    }
    else
    {

```

```

        return fibonacci(n-1)+fibonacci(n-2);
    }
}

```

1.5. Lección 4

Resolver los siguientes ejercicios usando la API de Tratamiento de Cadenas de Caracteres que desarrollamos durante el capítulo 2.

1.5.1. Valor numérico asociado a una cadena

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Se ingresa por teclado una cadena de caracteres, sin espacios en blanco y totalmente en mayúscula. Considerando que cada uno de sus caracteres tiene un valor numérico según la siguiente lista: A=1, B=2, C=3, D=4, ...M=13, N=14, Ñ=15, O=16, ... así hasta llegar a Z=27, se debe obtener la suma de los valores asignados a cada uno de los caracteres de la cadena ingresada. Si la suma obtenida tiene más de un dígito, habrá que sumar sus dígitos. Así sucesivamente hasta obtener un valor numérico de un único dígito.

Por ejemplo: Σ OCTAVIANO = 104. Como este valor tiene más de un dígito sumamos sus dígitos: $1 + 0 + 4 = 5$. Dado que 5 es un número de un único dígito, llegamos al resultado final. De otro modo, hubiéramos tenido que sumar sus dígitos una y otra vez hasta obtener un valor de un solo dígito.

NOTA: La cadena ingresada no tendrá caracteres especiales como Ñ o tildes.

1.5.2. Primeros n números que tienen m dígitos d

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Se ingresan por teclado 3 valores enteros: n , m y d . Se pide mostrar por consola los primeros n números naturales que tienen, al menos, m dígitos d . Por ejemplo, si $n=5$, $m=3$ y $d=4$, la salida del programa debería ser: 444, 1444, 2444, 3444, 4044.

1.5.3. Anagrama

Dificultad: **moderada**, *Requerido:* **Indispensable**.

Desarrollar y probar adecuadamente la función `esAnagrama` (según el siguiente prototipo), que retorna `true` o `false` según resulte que las dos cadenas de recibe sean o no anagramas entre sí.

```
bool esAnagrama(string a,string b);
```

1.5.4. Interpretar datos contenidos en una cadena

Dificultad: **intermedia**, *Requerido:* **recomendable**.

Se ingresa por teclado una cadena que contiene el nombre, la fecha de nacimiento y la nacionalidad de varias personas. Por ejemplo:

"Pedro,2-oct-1970,Argentino|Juan,9-dic-1985,Chileno|Pablo,14-ene-1992,Argentino"

Como vemos, la cadena contiene los datos de Pedro, Juan y Pablo separados mediante el carácter '|' (carácter pipe). A su vez, los datos de cada uno de ellos se separan entre sí mediante el carácter ',' (carácter coma). Por su parte, los atributos de las fechas de nacimiento están separados entre sí por el carácter '-' (guion).

Se pide desarrollar un programa que muestre por pantalla los datos de cada persona. Por ejemplo, según el ejemplo anterior, el programa debería mostrar:

```
Cantidad de personas: 3
---
Nombre: Pedro
Fecha de nacimiento: Dia 2, Mes oct, Año 1970
Nacionalidad: Argentino
---
Nombre: Juan
:
```

1.5.5. Reconocer si una cadena tiene un formato determinado

Dificultad: **intermedia**, *Requerido:* **recomendable**.

El usuario ingresa una cadena con su apellido, nombre, curso y legajo. Se pide validar que respete exactamente el siguiente formato: *Apellido(s), Nombre(s) (Curso, Legajo)*. Los apellidos pueden ser simples o compuestos y los nombres pueden incluir segundo nombre; ambos pueden abreviarse, pero toda abreviatura debe finalizar en punto. La coma tras el apellido y el espacio posterior son obligatorios. El curso siempre comienza con K mayúscula seguido de 4 dígitos. El legajo siempre consiste en 4 dí-

gitos. Curso y legajo se encierran entre paréntesis, separados por coma y espacio. Las mayúsculas y minúsculas deben conservarse tal como se escriben.

Algunos ejemplos válidos son:

- Sznajdleder, Pablo (K1001, 5432)
- Sznaj. Pablo Augusto (K1001, 5432)
- Sznajdleder R., Pablo (K1001, 5432)
- Sznajdleder R., Pablo August. (K1001, 5432)
- Sznajdleder Rotman, Pablo (K1001, 5432)

1.5.6. TAD BigInt

Dificultad: **intermedia**, *Requerido:* **recomendable**.

Desarrollar y probar el TAD `BigInt` de acuerdo con la estructura y especificaciones de las funciones que se describen a continuación.

Nombre del TAD: `BigInt`. *Archivo:* `BigInt.hpp`

Descripción: Representa números enteros muy grandes, sin restricciones de tamaño ni cantidad de dígitos.

```
struct BigInt
{
    string s;
};
```

Prototipo: `BigInt bigInt(string n);`

Descripción: Crea (o instancia) un `BigInt`.

Parámetro: `string n` - Cadena que contiene la representación del número.

Retorna: `BigInt` - Una instancia de `BigInt` lista para trabajar con el número `n`.

Prototipo: `BigInt bigIntSumar(BigInt a, BigInt b);`

Descripción: Retorna una instancia `BigInt` que contiene la suma de `a` y `b`.

Parámetros:

- `BigInt a` - Valor a sumar.
- `BigInt b` - Valor a sumar.

Retorna: `BigInt` - Una instancia de `BigInt` que contiene la suma de `a` y `b`.

Prototipo: `BigInt bigIntRestar(BigInt a, BigInt b);`

Descripción: Retorna una instancia `BigInt` que contiene la resta de `a` y `b`.

Parámetros:

- `BigInt a` - Minuendo.
- `BigInt b` - Sustraendo.

Retorna: `BigInt` - Una instancia de `BigInt` que contiene la resta de `a` menos `b`.

1.5.7. TAD Matriz

Dificultad: [intermedia](#), *Requerido:* [recomendable](#).

Desarrollar y probar el TAD Matriz según la siguiente estructura y especificaciones.

Nombre del TAD: Matriz.

Descripción: Representa una matriz numérica de n filas y m columnas.

Restricción: Cada celda de la matriz puede contener un número de un único dígito.

```
struct Matriz
{
    string s;
    int filas;
    int columnas;
}
```

Prototipo: `Matriz matriz(int n, int m);`

Descripción: Retorna una instancia Matriz.

Parámetros:

- `int n` - Cantidad de filas de la matriz.
- `int m` - Cantidad de columnas de la matriz.

Retorna: `Matriz` - Una instancia de `Matriz` preparada para contener una matriz de `n` filas y `m` columnas.

Prototipo: `Matriz matriz(String x,int n,int m);`

Descripción: Retorna una instancia `Matriz` inicializada con el contenido de `x`.

Parámetros:

- `String x` - Contenido de la matriz que se está instanciando.
- `int n` - Cantidad de filas.
- `int m` - Cantidad de columnas.

Retorna: `Matriz` - Una instancia de `Matriz` de `n` filas y `m` columnas, inicializada con el contenido de la cadena `x`.

Ejemplo: Si `x="123456789012"`, `n=3` y `m=4`, la matriz representada por este conjunto de parámetros que estos parámetros están representando es la siguiente:

1	2	3	4
5	6	7	8
9	0	1	2

Prototipo: `int matrizGet(Matriz m,int f,int c);`

Descripción: Retorna el valor (número de un dígito) que la matriz `m` contiene en la celda determinada por la fila `f` y la columna `c`.

Parámetros:

- `Matriz m` - Matriz.
- `int f` - Fila.
- `int c` - Columna.

Retorna: `int` - El valor (número de un dígito) que la matriz `m` contiene en la celda determinada por la fila `f` y la columna `c`.

Prototipo: `void matrizSet(Matriz& m,int f,int c,int v);`

Descripción: Asigna el valor v (número de un dígito) en la celda determinada por la fila f y la columna c de la matriz m .

Parámetros:

- `Matriz m` - Matriz.
- `int f` - Fila.
- `int c` - Columna.
- `int v` - Valor numérico (de un solo dígito) que se asignará.

Retorna: `void`.

Prototipo: `Matriz matrizSumar(Matriz a, Matriz b);`

Descripción: Retorna la matriz que resulta de sumar $a+b$.

Parámetros:

- `Matriz a` - Matriz a sumar.
- `Matriz b` - Matriz a sumar.

Retorna: `Matriz` - La matriz resultante de la suma de las matrices a y b .

Prototipo: `Matriz matrizRestar(Matriz a, Matriz b);`

Descripción: Retorna la matriz que resulta de restar $a-b$.

Parámetros:

- `Matriz a` - Minuendo.
- `Matriz b` - Sustraendo.

Retorna: `Matriz` - La matriz que resulta de restar $a-b$.

NOTA: Esta implementación del TAD `Matriz` tiene la restricción de que cada elemento de la matriz debe ser un valor numérico de un solo dígito, sin signo. Por esto, aceptaremos que las operaciones de suma y resta no arrojarán resultados que excedan estas limitaciones.

Por ejemplo: si sumamos las dos matrices que vemos a continuación, la matriz resultante cumple con las restricciones del TAD.

1	4	2	6	5	3
7	2	5	1	2	4
1	3	8	2	6	1

Figura 2.2. Ejemplo de matrices que podrían aplicar para la operación matrizSumar

1.6. Lección 5

Los siguientes ejercicios deben resolverse usando las funciones de las bibliotecas de *tokens* y cadenas de caracteres.

En todos los casos, salvo que se indique lo contrario, se debe considerar que el usuario ingresará por teclado un conjunto de valores numéricos enteros, finalizando el ingreso de datos con un valor 0 (cero), que no formará parte del conjunto.

1.6.1. Invertir el orden de un conjunto

Mostrar los elementos del conjunto en el orden inverso al que fueron ingresados.

1.6.2. Elementos repetidos

Mostrar todo el conjunto descartando los valores repetidos, si los hubiera.

1.6.3. Posiciones del máximo y mínimo

Indicar en qué posiciones se encuentra el máximo y el mínimo valor. Por ejemplo, si se ingresa: 3, 6, 1, 8, -1, 2, 8, 4, -1, 5, la salida debería ser: Máximo: 8, en posiciones: {3, 6}, Mínimo: -1, en posiciones: {4, 8}.

1.6.4. Suma

Mostrar todos los pares, tal la suma de sus componentes sea 30. Por ejemplo, si el conjunto ingresado fuera: 3, 25, 8, 2, 5, 19, 22, 9, 14, 11, 4, la salida debería ser:

```
25+5 = 30
8+22 = 30
19+11 = 30
```

1.6.5. Ordenar un conjunto de valores

Mostrar todos los elementos del conjunto ordenados de mayor a menor, y luego mostrarlos ordenados menor a mayor.

1.6.6. Medir cuánto tiempo demora ordenar un conjunto de tokens

- Desarrollar la función `tokenSort(string& s, char sep)`; que ordena alfabéticamente ascendente los *tokens* de la cadena *s* considerando a *sep* como separador.
- Usando la función `randomString` de la biblioteca de cadenas de caracteres, generar una cadena *tokenizada* con 10 tokens, ordenarla e indicar cuánto tiempo demoró el proceso.
- Idem anterior, con 100 y con 1000 *tokens*.

1.6.7. Rotar un conjunto de valores

Además del conjunto de valores numéricos enteros, el usuario ingresará un valor *k* que llamaremos *valore de rotación*. Se pide mostrar todos los elementos del conjunto rotándolos *k* posiciones. Por ejemplo, si el conjunto fuera: 1, 2, 3, 4, 5 y *k*=3, la salida deberá ser: 3, 4, 5, 1, 2.

Es decir, en el ejemplo anterior el último valor es 5 y el primero es 1. Entonces, sacamos el último valor y lo colocamos en el primer lugar, desplazando todos los demás valores hacia la derecha. Así *k* veces.

1.6.8. Subconjunto consecutivo de suma máxima

El usuario ingresa un conjunto de valores numéricos, se pide encontrar la mayor suma posible de elementos consecutivos. Por ejemplo:

- Si el conjunto fuera: 2, -1, 3, -2, 4, el mejor subconjunto será todo el conjunto y sumaría 6.
- Si fuera: -3, 5, -1, 2, -4, el mejor subconjunto sería: 5, -1, 2 y sumaría 6.
- Y si fuese: 1, 2, -5, 4, 3, el mejor subconjunto sería: 4, 3 y sumaría: 7.

1.7. Lección 7

Repetir todos los ejercicios de la lección 5 usando el TAD `Coll`.